AI Scenario Based Language Learning App: Implementation Plan

Languages at launch: Kannada and English (mutual) **Stack:** Flutter (frontend) + Python (backend) + Supabase (auth/db) + on device LLM (free tier) + cloud LLM (paid tier)

---

1. Product Summary

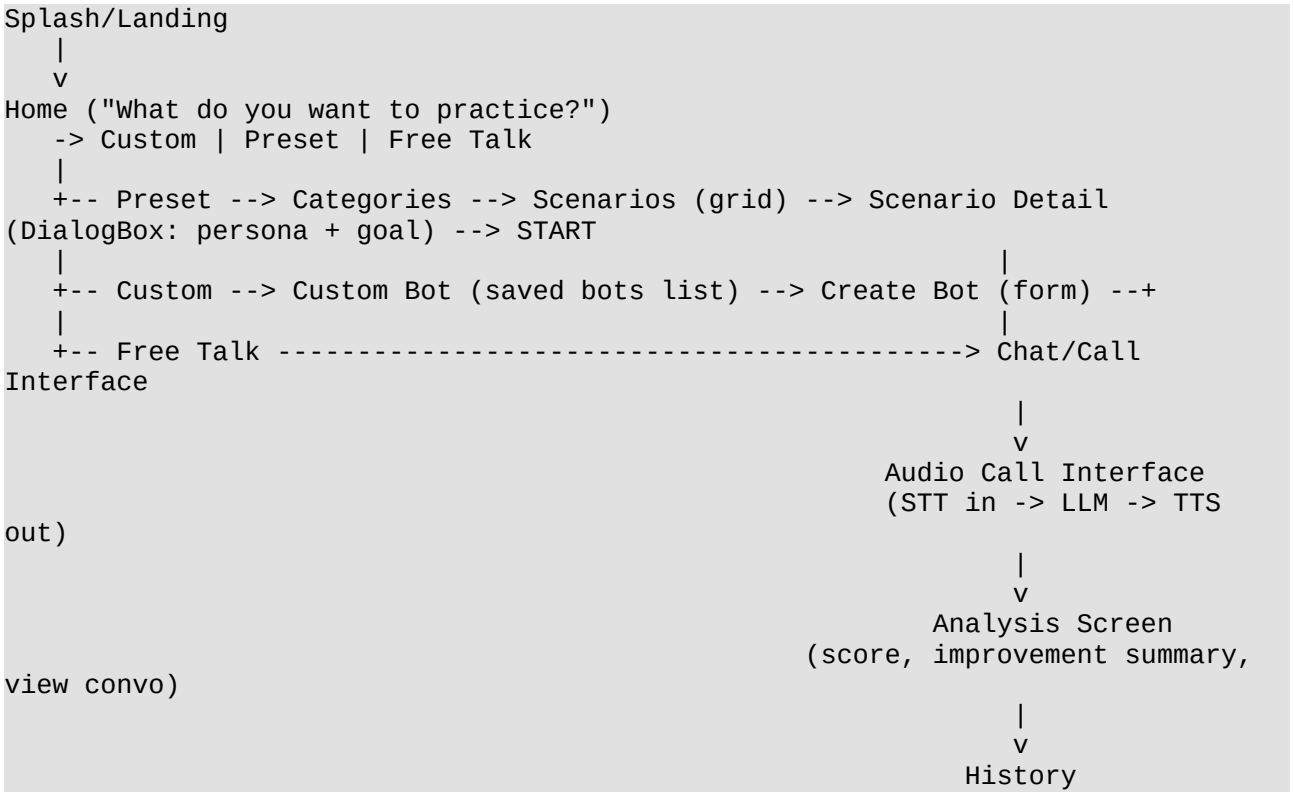
A scenario based conversational app where the user picks or creates a roleplay scenario (e.g. "order coffee in Bengaluru," "ask for directions," "interview for a job") and has a text/voice conversation with an AI character who stays in that role. After the conversation, the app scores the attempt and gives improvement notes.

Two tiers:

- **Free**: on device small model (Gemma 3 1B or similar), works offline, capped scenario library.
- **Premium**: cloud model via an OpenAI compatible API, full scenario library, better feedback quality, voice in both languages.

---

2. Screen Flow (from your wireframe)



Screens to build, in order of build priority:

1. Auth (Google OAuth via Supabase)

2. Home
3. Categories -> Scenarios -> Scenario Detail
4. Chat interface (text first)
5. Custom Bot list + Create Bot form
6. Analysis screen
7. History
8. Audio call interface (STT/TTS layer on top of chat)
9. Paywall / upgrade screen (not in wireframe but required for premium gating)

---

3. Architecture Overview

```
Flutter App
  |-- Supabase SDK (auth, Postgres, storage, realtime)
  |-- Local inference engine (llama.cpp via flutter binding, e.g. fllama or
llama_cpp_dart)
  |-- HTTP client -> Python Backend (for premium chat, TTS/STT, analysis,
scenario sync)
  |
Python Backend (FastAPI)
  |-- /auth        -> verifies Supabase JWT
  |-- /chat         -> routes to cloud LLM (premium) or returns local-mode
config
  |-- /scenario     -> CRUD for preset + custom scenarios
  |-- /stt          -> AI4Bharat IndicConformer (Kannada) / Whisper (English)
  |-- /tts          -> AI4Bharat Indic-Parler-TTS or IndicF5 (Kannada) / Piper
or Azure (English)
  |-- /analysis     -> scores conversation, generates summary
  |-- /billing      -> webhook handler for subscription provider
  |
Supabase
  |-- Postgres: users, scenarios, custom_bots, conversations, messages,
subscriptions
  |-- Auth: Google OAuth
  |-- Storage: user-uploaded avatars, generated audio cache (optional)
```

Why this split: the free tier must work with zero backend cost beyond auth, so on-device inference is the load-bearing piece for free users. The backend exists mainly to serve premium users and to do anything that genuinely needs a server (Kannada STT/TTS, scoring, scenario sync, payments).

---

4. Model Strategy

4.1 Free tier (on-device)

Target: phones with 4-6GB RAM. At this tier you have real headroom problems: the OS already takes 2-4GB, so you're budgeting for roughly 2-3GB of usable space for model + app + context.

Model	Params	Size (Q4_K_M)	Fit on 4-6GB phone
Gemma 3 1B	1B	~720MB	Yes, comfortably, all 4-6GB devices
Qwen 2.5 1.5B	1.5B	~1.0GB	Yes, good multilingual support
SmolLM 2 1.7B	1.7B	~1.1GB	Yes, fastest of the larger ones
Gemma 3 4B / Qwen 3 4B	4B	~2.9GB	Risky on 4GB phones, fine on 6GB

Recommendation: ship **Gemma 3 1B** as the default free-tier model, with **Qwen 2.5 1.5B** as an optional "better quality, slightly slower" toggle for users on 6GB+ devices. Reasons:

- Gemma 3 1B is the smallest viable model and the safest default across your whole target range (you said 4-6GB, not 8GB+).
- Qwen 2.5 1.5B has stronger multilingual behavior, which matters since you're doing Kannada ↔ English, not just English.
- Both run via `llama.cpp`-based engines on Flutter (e.g. `fllama`, `llama_cpp_dart`) using GGUF format, no server round trip, fully offline.

Detect device RAM at runtime and pick the model tier automatically; let the user override in settings.

Important constraint: neither model is strong at Kannada generation out of the box (small models lean English-heavy). For free tier, the realistic v1 scope is: **conversation in English, with Kannada vocabulary/phrase practice using a fixed phrase bank** rather than freeform Kannada generation from the small model. Freeform Kannada roleplay is a premium-tier feature where you can afford a bigger cloud model. Flag this to your team now, since it changes the free tier's pitch from "full bilingual AI conversation" to "AI conversation practice (English) + structured Kannada phrase drills," which is still a legitimate free tier, just not identical in capability to premium.

4.2 Premium tier (cloud)

Use an OpenAI-compatible API so you can swap providers without rewriting client code. Options, roughly in order of capability-per-cost for a conversational/roleplay use case:

- A frontier-class hosted model via an OpenAI-compatible endpoint (best Kannada generation quality, highest cost).
- A hosted open-weight model (e.g. Qwen 3 7B+ class or Llama 3.3 8B) through an inference provider like Together.ai, Groq, or Fireworks: cheaper per token, still OpenAI-compatible, noticeably better multilingual handling than the 1-2B on-device models.

Build your backend's `/chat` endpoint against the OpenAI `chat.completions` schema so either choice is a config change, not a code change.

4.3 STT / TTS

This is the part most likely to surprise you on cost/complexity, so scope it deliberately:

- **Kannada STT:** AI4Bharat's IndicConformer (Conformer-Large, CTC-RNNT, MIT license), self-hostable on your VPS, no per-call cost, decent latency (sub-100ms claimed by some wrappers, but verify on your own hardware).
- **Kannada TTS:** AI4Bharat's Indic-Parler-TTS or IndicF5, also self-hostable, supports Kannada with emotion/tone control. Quality is good but not at ElevenLabs/Azure level; treat it as "free tier" voice quality.
- **English STT:** Whisper (small/base, self-hosted) is more than sufficient and well trodden.
- **English TTS:** Piper (fully offline, tiny, decent quality) for free tier; consider Azure Neural TTS for premium English voice if you want a noticeably better voice and don't mind per-character billing.
- **Premium voice upgrade path:** if budget allows later, Azure Speech has the strongest Indic language coverage commercially (60+ Indic neural voices including Kannada) and is the standard recommendation if self-hosted AI4Bharat quality isn't good enough for paying users.

Run STT/TTS as separate microservices (or at least separate processes) behind your FastAPI backend so you can scale or swap them independently of the chat logic.

---

5. Backend (Python)

Framework: FastAPI (async, OpenAPI docs for free, plays well with Supabase's Python client and httpx for outbound LLM calls).

Core endpoints

Endpoint	Method	Purpose
`/v1/auth/verify`	POST	Validate Supabase JWT on protected routes (middleware, not really a user-facing endpoint)
`/v1/scenarios`	GET	List preset scenarios (with category filter)
`/v1/scenarios/{id}`	GET	Scenario detail (persona, goal, opening line)
`/v1/bots`	GET/POST	List/create custom bots for the user
`/v1/bots/{id}`	PUT/DELETE	Edit/delete a custom bot
`/v1/chat/completions`	POST	Premium-tier chat, OpenAI-compatible passthrough with system prompt injection for persona

Endpoint	Method	Purpose
`/v1/stt`	POST	Audio in, transcript out (lang param: kn/en)
`/v1/tts`	POST	Text in, audio out (lang param: kn/en)
`/v1/analysis`	POST	Takes a full conversation, returns score + improvement summary
`/v1/history`	GET	Past conversations for the user
`/v1/billing/webhook`	POST	Subscription provider webhook (Razorpay/Stripe)

Key design decisions

- **Stateless chat endpoint:** client sends full message history each call (same pattern as the OpenAI API itself); backend doesn't hold conversation state in memory. Conversation state lives in Supabase Postgres, written by the client after each turn (or by the backend if you want server-side persistence as the source of truth, which is recommended, since it also lets Analysis read directly from Postgres instead of trusting client-submitted transcripts).
- **System prompt construction:** for each scenario/bot, store a persona template in Postgres (`system_prompt`, `goal`, `opening_line`). Backend assembles the final system prompt server-side so you can update persona behavior without an app release.
- **Rate limiting:** apply per-user limits on `/chat/completions` for premium users too (protects you from cost blowouts), and a stricter cap for any server-assisted free-tier features.

---

6. Database Schema (Supabase Postgres)

```
-- users handled by Supabase Auth (auth.users), extend with a profile table
create table profiles (
  id uuid primary key references auth.users(id),
  display_name text,
  native_language text check (native_language in ('kn','en')),
  learning_language text check (learning_language in ('kn','en')),
  subscription_tier text default 'free' check (subscription_tier in
('free','premium')),
  device_ram_mb int,
  created_at timestamptz default now()
);

create table categories (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  sort_order int default 0
);

create table scenarios (
  id uuid primary key default gen_random_uuid(),
```

```
  category_id uuid references categories(id),
  title text not null,
  description text,
  system_prompt text not null,
  goal text,
  opening_line text,
  difficulty text check (difficulty in ('beginner','intermediate','advanced')),
  is_premium boolean default false
);

create table custom_bots (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid references profiles(id),
  name text not null,
  system_prompt text not null,
  goal text,
  created_at timestamptz default now()
);

create table conversations (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid references profiles(id),
  scenario_id uuid references scenarios(id),
  bot_id uuid references custom_bots(id),
  started_at timestamptz default now(),
  ended_at timestamptz,
  score numeric,
  improvement_summary text
);

create table messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid references conversations(id),
  role text check (role in ('user','assistant')),
  content text not null,
  created_at timestamptz default now()
);

create table subscriptions (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid references profiles(id),
  provider text,
  provider_subscription_id text,
  status text,
  current_period_end timestamptz
);
```

Enable Row Level Security on every user-owned table (`profiles`, `custom_bots`, `conversations`, `messages`, `subscriptions`) so users can only read/write their own rows; do this from day one, not as a retrofit.

---

7. Flutter App Structure

```
lib/
  main.dart
  core/
    supabase_client.dart
    api_client.dart          // talks to FastAPI backend
    local_llm/
      model_manager.dart     // downloads, verifies, switches GGUF models
```

```
      inference_engine.dart  // wraps fllama / llama_cpp_dart
    stt_tts/
      stt_service.dart       // routes to local Whisper.cpp or backend /stt
      tts_service.dart       // routes to local Piper or backend /tts
  features/
    auth/
    home/
    scenarios/               // categories, scenario grid, scenario detail
    bots/                    // custom bot list, create bot form
    chat/                    // text chat screen + call screen
    analysis/
    history/
    paywall/
  shared/
    widgets/
    models/                  // Scenario, Bot, Conversation, Message
state management: Riverpod (recommended) or Bloc, pick one, don't mix
```

On-device inference library: use `fllama` (Flutter binding around llama.cpp, supports GGUF, cross-platform) or `llama_cpp_dart` as a more minimal alternative. Both let you ship a model file, load it, and stream tokens back to the UI.

Model distribution: don't bundle the GGUF in the app binary (it'll bloat the install size and app store review gets fussy over large binaries). Download on first launch from your own storage (Supabase Storage or a CDN) with a progress screen, cache locally, and let users delete/re-download from settings.

---

8. Premium / Paywall Logic

- Gate by `profiles.subscription_tier` checked server-side on every premium-only endpoint (`/chat/completions`, premium scenarios, premium voice); never trust a client-side flag alone.
- Payment provider: **Razorpay**. Handles UPI, cards, netbanking, and wallets, which covers an India-heavy user base well. Use Razorpay Subscriptions (not one-off orders) so renewals are handled by Razorpay rather than you re-billing manually. Webhook into `/v1/billing/webhook` to flip `subscription_tier` on `subscription.activated`, `subscription.charged`, and `subscription.cancelled` events. Verify the webhook signature server-side before trusting any payload.
- Free tier caps to enforce: limited scenario count unlocked, no custom bot creation (or a cap like 1-2), English-only conversation (Kannada via phrase bank, per section 4.1), no voice call mode (text only), or a very limited number of voice minutes/day if you want to give a taste.

---

9. Analysis / Scoring

After a conversation ends:

1. Backend pulls the full message history for that `conversation_id` from Postgres.
2. Sends it to the cloud LLM (even for free-tier users, since this one step can run server-side since it's infrequent, not per-message) with a scoring prompt: goal completion, grammar, vocabulary range, fluency.
3. Store `score` and `improvement_summary` back on the `conversations` row.
4. Analysis screen reads these plus a "View convo" link back into the transcript.

This keeps scoring quality high (it's the one place where using a bigger model for everyone, even free users, is worth the small server cost, since it only fires once per session, not per message).

---

10. Build Order (Suggested Milestones)

1. **Foundation:** Supabase project, Google OAuth, profile creation, DB schema + RLS.
2. **Backend skeleton:** FastAPI with auth middleware, scenarios CRUD, deployed to VPS behind nginx + HTTPS (Let's Encrypt).
3. **Core chat loop (text only, English, on-device model):** Home -> Scenario -> Chat -> end conversation -> save to Postgres. This is your first demoable slice.
4. **Custom bots:** Create Bot form -> Custom Bot list -> chat with custom persona.
5. **Analysis screen:** wire up scoring after conversation end.
6. **History screen.**
7. **Premium tier:** cloud LLM routing, paywall, billing webhook.
8. **Voice layer:** STT/TTS services on VPS, call interface UI, wire into existing chat logic (the LLM/persona layer shouldn't need to change, just the input/output modality).
9. **Kannada phrase bank (free tier)** and **Kannada freeform chat (premium tier)**.
10. **Polish:** model auto-selection by device RAM, offline mode handling, error states, onboarding.

---

11. Roadmap

This extends the build order in Section 10 into a phased release roadmap, end to end from the wireframe to a live, monetized product.

Phase 0: Foundation (pre-UI)

- Finalize Supabase schema + RLS policies.
- Stand up VPS: nginx, HTTPS, FastAPI skeleton, deployment pipeline (even a simple `git pull` + `systemctl restart` script is fine at this stage).
- Google OAuth working end to end (Flutter -> Supabase -> session persisted).

Phase 1: Core Loop (matches wireframe screens: Home, Categories, Scenarios, Scenario Detail, Chat)

- Home screen with Custom / Preset / Free Talk entry points.
- Categories -> Scenarios grid -> Scenario Detail (DialogBox showing persona + goal).
- Text-only chat screen wired to the on-device model (Gemma 3 1B default).
- Conversation persisted to Postgres (`conversations`, `messages`).
- **Exit criteria:** a user can open the app, pick a preset scenario, have a full English text conversation offline, and have it saved.

Phase 2: Custom Bots (matches wireframe: Custom Bot, Create Bot)

- Custom Bot list screen.
- Create Bot form (name, persona/system prompt, goal).
- Chat screen reused against custom bot personas instead of presets.
- Free-tier cap enforced (e.g. 1-2 custom bots).
- **Exit criteria:** a user can build their own bot and chat with it, same engine as Phase 1.

Phase 3: Analysis & History (matches wireframe: Analysis, History)

- Backend `/v1/analysis` endpoint: pulls transcript, scores via cloud LLM, writes back `score` + `improvement_summary`.
- Analysis screen: score display, improvement summary, "View convo" link.
- History screen: list of past conversations, tap through to transcript.
- **Exit criteria:** every finished conversation produces a score and shows up in history.

Phase 4: Monetization

- Razorpay Subscriptions integration (plan creation in Razorpay dashboard, checkout flow in Flutter via Razorpay's Flutter SDK).
- `/v1/billing/webhook` handling activation/charge/cancellation events, flipping `profiles.subscription_tier`.
- Paywall screen + upgrade prompts at natural friction points (locked scenario, custom bot cap reached, voice mode).
- Cloud LLM routing for premium `/chat/completions` (OpenAI-compatible provider of choice).
- **Exit criteria:** a user can pay via Razorpay, get flipped to premium server-side, and immediately get a better model + unlocked content.

Phase 5: Voice (matches wireframe: Any good audio call interface)

- Backend `/v1/stt` and `/v1/tts` services: Whisper for English, AI4Bharat

IndicConformer/Indic-Parler-TTS for Kannada.

- Call interface UI: record -> STT -> existing chat pipeline -> TTS -> playback.
- Free tier: text only, or a small daily voice-minute allowance if you want to give a taste.
- Premium tier: full voice access, optionally routed to Azure Neural TTS later if self-hosted quality isn't good enough for paying users.
- **Exit criteria:** the audio call screen from the wireframe works end to end in both languages for premium users.

Phase 6: Kannada Depth

- Free tier: structured Kannada phrase bank (vocabulary/phrase drills, not freeform generation; see Section 4.1 for why).
- Premium tier: freeform Kannada roleplay via the cloud model.
- Revisit whether a Kannada-tuned small model can eventually close this gap for free tier (tracked in Section 12).

Phase 7: Polish & Scale

- Automatic model tier selection by device RAM, with manual override in settings.
- Offline mode handling (graceful degradation when premium features need connectivity and it's unavailable).
- Onboarding flow, error states, empty states across all screens.
- Usage analytics, crash reporting, and basic admin tooling for managing scenarios/categories without a redeploy.

---

12. Open Decisions to Revisit

- Whether free-tier Kannada stays phrase-bank-only long term, or whether a Kannada-tuned small model becomes good enough to flip it to freeform later.
- Whether to self-host AI4Bharat STT/TTS indefinitely or migrate premium voice to Azure once you have paying users to justify the cost.